Assignment 2: From Trees to Neural Networks

1. Introduction

This report focuses primarily on two machine learning models: Gradient Boosted Decision Trees and Multi Layer Perceptrons. These two models applied to a binary classification task in credit risk assessment. The goal is to predict whether a loan applicant is likely to default, using the Home Credit Default Risk dataset from Kaggle.

Many people struggle to get loans due to insufficient or non-existent credit histories. And, unfortunately, this population is often taken advantage of by untrustworthy lenders. Predicting loan default is therefore an important task. The dataset has 307511 applications and 122 features covering demographics, financials, and external credit scores. One challenge is that the classes are heavily imbalanced. About 91.9% of applicants repaid, and only 8.1% are defaulters. A model that blindly predicts "repay" for everyone would score 92% accuracy while being completely useless.

2. Methods

Data Preparation

Roughly 49 columns had over 40% of their values missing. Indicating that much data are unreliable at that point, they are also not worth replacing with mean value since too many rows are missing. The ID column SK_ID_CURR was also removed. That left 72 features to work with. For the remaining missing values, numeric columns were filled with medians (more robust to outliers than means), and categorical columns got the training set mode. All imputation statistics came from the training set only, I was careful about avoiding data leakage. Encoding

was split by cardinality. ORGANIZATION_TYPE has 58 categories, so one-hot encoding would have exploded the feature space with sparse columns. Label encoding therefore makes more sense there. The other 11 categorical features had fewer than 20 categories each and were one-hot encoded with drop_first=True. Final feature count: 115.

The data was split 70/15/15 into train, validation, and test sets using stratified sampling to keep the class ratio consistent. A separate scaled copy of the data was created for MLP using StandardScaler fit on training data only. Scaling matters for MLP because gradient descent struggles when features live on wildly different scales, income in the hundreds of thousands versus binary features creates an imbalanced gradient. Trees don't care about scale since they just split on thresholds within each feature. For class imbalance, XGBoost's scale_pos_weight was set to ~11.3. MLPClassifier doesn't offer built-in class weighting, so decision threshold tuning was used instead, picking the threshold that maximized F1 on the validation set rather than defaulting to 0.5.

GBDT

XGBoost was trained on the unscaled features with a baseline configuration: **learning_rate=0.1, max_depth=6, n_estimators=500, and subsample=0.8**. Both training and validation log loss were tracked through eval_set, with early stopping patience set to 50 rounds to catch overfitting before it got out of hand. To understand how learning rate shapes the training dynamics, three values were compared: 0.01, 0.1, and 0.3. Each represents a different thing: small steps that generalize well but train slowly, versus large steps that converge fast but risk overshooting. The loss curves for each tell a clear story about this trade-off.

A broader search followed using RandomizedSearchCV (20 iterations, 3-fold CV, optimizing AUC-ROC). The search covered max_depth (3–8), subsample (0.6–1.0), reg_alpha and reg_lambda (0–10), learning_rate (0.01–0.1), and n_estimators (300–500). The winning configuration turned out more conservative than the baseline in almost every way: shallower trees (max_depth=3), a gentler learning rate (lr=0.05), and notably strong L1 regularization (reg_alpha=10).

MLP

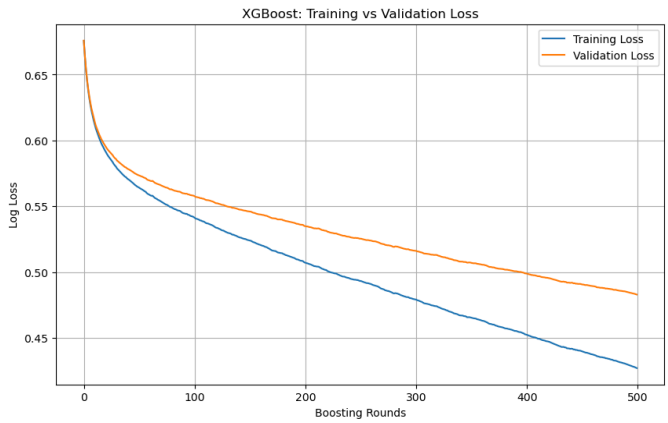
MLPClassifier was trained on the standardized features. The baseline used two hidden layers (128, 64) with ReLU activation, learning_rate_init=0.001, and early stopping with n_iter_no_change=30. One early finding: without any class imbalance handling, the initial model predicted only 3 out of 46,127 validation samples as defaulters. It had essentially learned to say "repay" to everyone. Threshold tuning can fix this, but it highlights a real limitation of sklearn's MLP compared to XGBoost's built-in scale_pos_weight.

Three architectures were tested to examine how depth and width affect learning: a single layer (64), two layers (128, 64), and three layers (256, 128, 64). The question was whether adding capacity would help the model capture more complex patterns or just overfit to training noise.

Activation functions were compared between ReLU and tanh. ReLU is the default, it's computationally cheaper and avoids vanishing gradients in deeper networks, but tanh can sometimes work better on smaller architectures. Finally, three learning rates (0.001, 0.01, 0.1)

were tested. Neural networks tend to be far more sensitive to this parameter than tree-based models, and the results confirmed that.

3. Results



Training vs Validation Loss without tuning

GBDT Results

- **Baseline vs Tuned Performance**

The baseline XGBoost model's AUC-ROC score is 0.7426 on the validation set. After hyperparameter tuning, this improved to 0.7523. The baseline also showed training loss of about 0.43 versus validation loss of 0.48 by round 500, which is a clear sign of overfitting. The tuned

model narrowed the gap to 0.574 vs 0.579. Shallower trees and stronger regularization helped.

```
[0]     validation_0-logloss:0.67568    validation_1-logloss:0.67588
[50]    validation_0-logloss:0.56410    validation_1-logloss:0.57339
[100]   validation_0-logloss:0.54131    validation_1-logloss:0.55765
[150]   validation_0-logloss:0.52401    validation_1-logloss:0.54606
[200]   validation_0-logloss:0.50708    validation_1-logloss:0.53485
[250]   validation_0-logloss:0.49326    validation_1-logloss:0.52541
[300]   validation_0-logloss:0.47911    validation_1-logloss:0.51611
[350]   validation_0-logloss:0.46526    validation_1-logloss:0.50695
[400]   validation_0-logloss:0.45233    validation_1-logloss:0.49882
[450]   validation_0-logloss:0.43964    validation_1-logloss:0.49058
[499]   validation_0-logloss:0.42709    validation_1-logloss:0.48293

Validation Results:
              precision    recall  f1-score   support

           0       0.95      0.78      0.86     42403
           1       0.18      0.56      0.28      3724

    accuracy                           0.76     46127
   macro avg       0.57      0.67      0.57     46127
weighted avg       0.89      0.76      0.81     46127

AUC-ROC: 0.7426
```

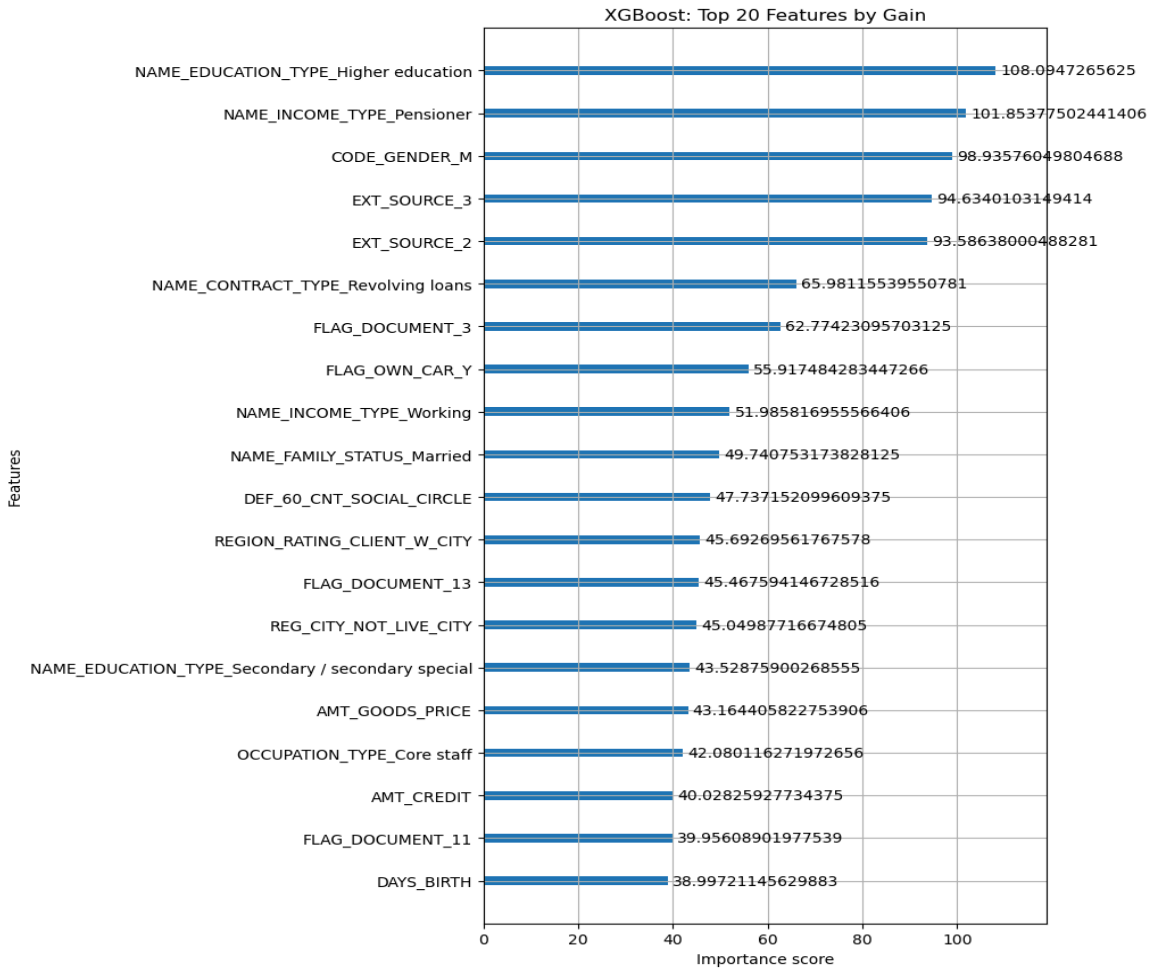
```
[0]     validation_0-logloss:0.68442    validation_1-logloss:0.68439
[50]    validation_0-logloss:0.61087    validation_1-logloss:0.61151
[100]   validation_0-logloss:0.59737    validation_1-logloss:0.59843
[150]   validation_0-logloss:0.59053    validation_1-logloss:0.59202
[200]   validation_0-logloss:0.58631    validation_1-logloss:0.58838
[250]   validation_0-logloss:0.58309    validation_1-logloss:0.58562
[300]   validation_0-logloss:0.58089    validation_1-logloss:0.58403
[350]   validation_0-logloss:0.57894    validation_1-logloss:0.58249
[400]   validation_0-logloss:0.57752    validation_1-logloss:0.58147
[450]   validation_0-logloss:0.57508    validation_1-logloss:0.57949
[499]   validation_0-logloss:0.57421    validation_1-logloss:0.57912

Tuned XGBoost - Validation Results:
              precision    recall  f1-score   support

           0       0.96      0.70      0.81     42403
           1       0.16      0.67      0.26      3724

    accuracy                           0.69     46127
   macro avg       0.56      0.68      0.53     46127
weighted avg       0.90      0.69      0.76     46127

AUC-ROC: 0.7523
AUC-PR: 0.2410
```

The recall score all increased from 0.56 to 0.67. The tuned model catches more actual defaulters. Precision dropped slightly (0.18 to 0.16), but in credit risk, missing a defaulter is usually more expensive than a false alarm.

- **Feature Importance**

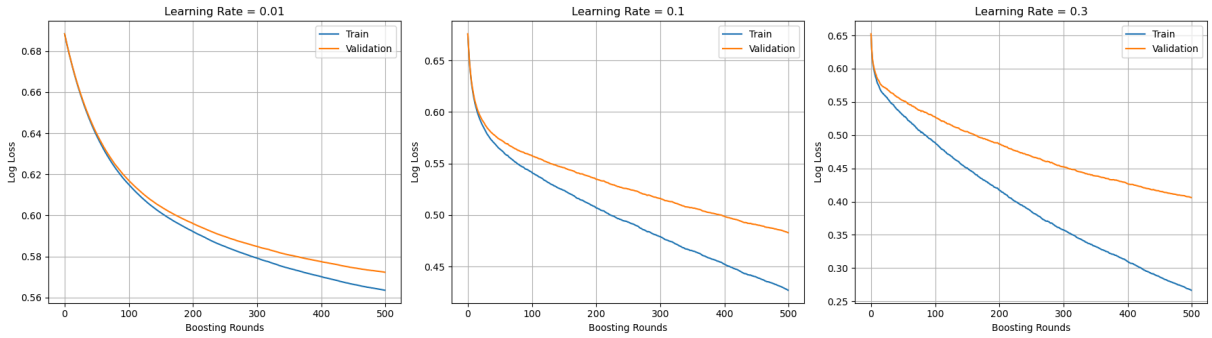
```
plot_importance(best_xgb, max_num_features=20, ax=ax, importance_type='gain')
```

The top features by gain (Figure X) were largely intuitive. Education type and income type ranked highest, followed by external credit scores (EXT_SOURCE_2 and EXT_SOURCE_3). Gender, contract type, and document flags also featured prominently. These results make sense, external credit assessments and socioeconomic indicators are important predictors of repayment behavior.



- **Learning Rate Comparison**

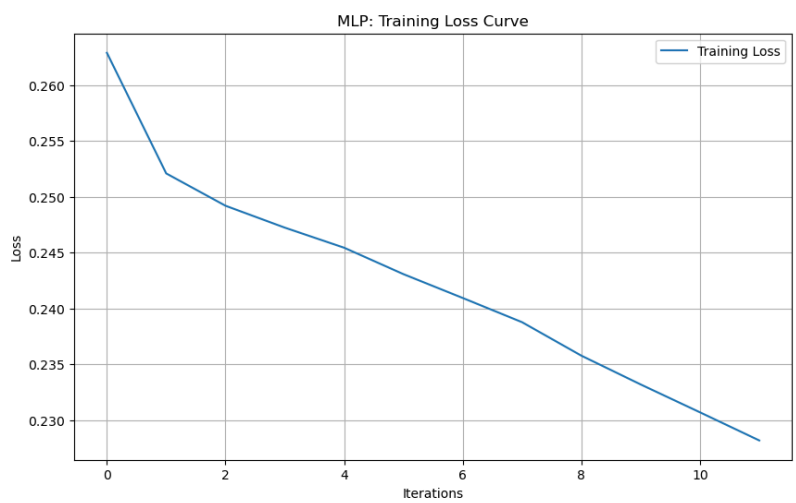
The three learning rate curves revealed distinct behaviors. First thing first, we need to be sure that we want something moderate, no gap between training and validation shows underfitting, and we also don’t want the log loss to be too high. In general, we don't want the gap to be zero (underfitting) or huge (overfitting). We want the lowest possible validation loss, with a gap that stays relatively stable rather than growing. At 0.01, both curves stayed close together but neither converged fully. At 0.1, the model reached a reasonable balance between speed and generalization. At 0.3, training loss plummeted to ~0.25 while validation loss flattened around 0.47, which is the widest gap of the three. The model was learning the training data fast but not generalizing.



MLP Results

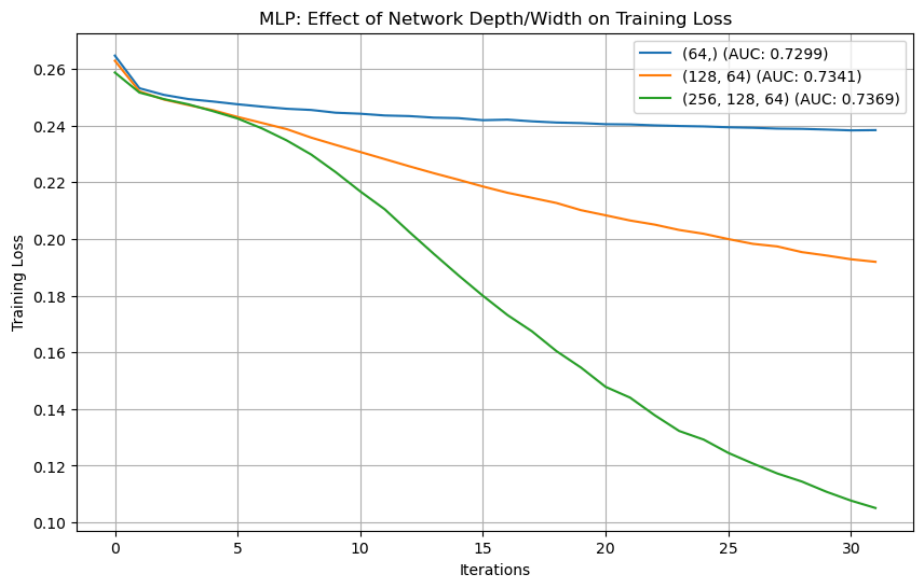
- **Baseline Performance/Training Loss Curve**

The MLP baseline achieved an AUC-ROC of 0.7341 with (128, 64) architecture. Training loss decreased, but early stopping happens after 32 iterations. Only 3 default predictions out of 46,127 samples. After optimizing the decision threshold on the validation set, predictions became more meaningful, though the AUC-PR of 0.2124 remained modest.



- **Effect of network depth/width on validation performance**

Wider and deeper networks brought diminishing returns. The single-layer network scored 0.7299, two layers scored 0.7341, and three layers scored 0.7369. Training loss dropped much more aggressively for the larger networks: the (256, 128, 64) model pushed loss down to about 0.10, but this extra capacity barely moved validation performance.



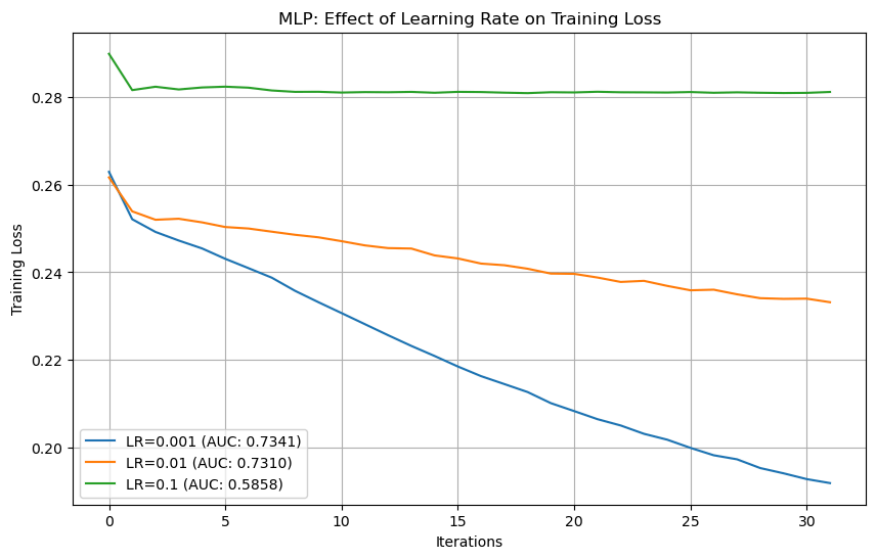
- **Activation Functions**

ReLU edged out tanh slightly (0.7341 vs 0.7317). Both converged in a similar number of iterations. Not a dramatic difference, but consistent with ReLU being the stronger default for this type of tabular data.

- **Learning Rate Sensitivity**

This was the most surprising result. At 0.001, the model trained smoothly and reached its best AUC of 0.7341. At 0.01, performance dipped slightly to 0.7310. At 0.1, the model failed completely. AUC collapsed to 0.5858 and the loss curve barely moved from its starting point.

Compare this to XGBoost, where a learning rate of 0.3 still produced a functional model. MLP's sensitivity here is a disadvantage obviously.



4. GBDT vs MLP Comparison & Discussion

```
=== GBDT vs MLP: Test Set Comparison ===
                    XGBoost (Tuned)  MLP (Best)
Accuracy                     0.7009      0.8318
Precision (Class 1)          0.1673      0.2168
Recall (Class 1)             0.6799      0.4146
F1 (Class 1)                 0.2685      0.2847
AUC-ROC                      0.7573      0.7382
AUC-PR                       0.2377      0.2133

XGBoost training time: 0.0796s
MLP training time: 56.51s
```

XGBoost had the stronger AUC-ROC (0.7573 vs 0.7382) and AUC-PR (0.2377 vs 0.2133), meaning it was better at ranking defaulters above non-defaulters overall. It also leaned heavily

toward recall, which caught 68% of actual defaulters but at the cost of many false alarms .MLP was more conservative, Accuracy tells a misleading story here. MLP's 83% looks far better than XGBoost's 70%, but that's because XGBoost's aggressive scale_pos_weight pushed it to predict more defaults, lowering accuracy while catching more true positives. In credit risk, missing a defaulter typically costs more, so XGBoost's trade-off is arguably more useful despite the lower accuracy number. Overall, GBDT is the stronger choice when you need fast training, built-in handling of missing values and class imbalance, or interpretable results. MLP makes more sense when the data has complex nonlinear interactions that trees can't capture.

From a bias-variance perspective, the two models fail in different ways. XGBoost's baseline overfit through deep trees memorizing training patterns, but regularization and shallower depth fixed this. MLP's main issue wasn't overfitting but rather underfitting the minority class, since it lacked built in mechanisms to handle imbalance. The threshold tuning workaround helped though.

5. AI Tool Disclosure

I used ChatGPT as a tool to check for the grammar and also explain some concepts I've been confused about. For example: I asked GPT to explain what hidden layers mean in MLP and the definition of AUC-ROC score. Another LLM I used was Claude, which helped me to make the visualization more neat and organized. I also consulted with it to know which python library I should use. The rest are my own work.